Radix-2 8-Point FFT (DIF) Hardware Implementation Report

Agamjot Singh
EE24BTECH11002

April 12, 2026

1 Introduction to DFTs

The discrete Fourier transform converts a finite-length discrete-time signal from the time domain to the frequency domain. For an N-point sequence $x[n]$, the DFT is

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-\mathrm{j}\frac{2\pi}{N}kn}, \qquad k = 0, 1, \ldots, N-1. \tag{1}$$

It is common to write the complex exponential using the twiddle factor

$$W_N = e^{-\mathrm{j}\frac{2\pi}{N}}, \tag{2}$$

so that (1) becomes

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn}. \tag{3}$$

For this report, the transform length is fixed at $N = 8$, hence

$$W_8 = e^{-\mathrm{j}\pi/4} = \frac{1-\mathrm{j}}{\sqrt{2}}, \tag{4}$$

and the useful powers are

$$W_8^0 = 1, \tag{5}$$

$$W_8^1 = \frac{1-\mathrm{j}}{\sqrt{2}}, \tag{6}$$

$$W_8^2 = e^{-\mathrm{j}\pi/2} = -\mathrm{j}, \tag{7}$$

$$W_8^3 = e^{-\mathrm{j}3\pi/4} = -\frac{1+\mathrm{j}}{\sqrt{2}}, \tag{8}$$

$$W_8^4 = -1. \tag{9}$$

2 Why FFT is Better than Direct DFT

2.1 Operation Count

A direct evaluation of the DFT in (3) computes each $X[k]$ using N complex multiplications and $N-1$ complex additions. Therefore,

$$\text{complex multiplications} = N^2, \tag{10}$$

$$\text{complex additions} = N(N-1). \tag{11}$$

For $N = 8$, this gives

$$\text{DFT multiplications} = 8^2 = 64, \tag{12}$$

$$\text{DFT additions} = 8(8-1) = 56. \tag{13}$$

The radix-2 FFT exploits repeated structure and symmetry in the twiddle factors to reduce the work. The canonical radix-2 complexity is

$$\text{FFT complex multiplications} \approx \frac{N}{2}\log_2 N, \tag{14}$$

$$\text{FFT complex additions} = N\log_2 N. \tag{15}$$

For $N = 8$,

$$\text{FFT multiplications} \approx \frac{8}{2}\log_2 8 = 12, \tag{16}$$

$$\text{FFT additions} = 8\log_2 8 = 24. \tag{17}$$

Many of those multiplications are by 1, -1, or $\pm\mathrm{j}$, which are essentially sign swaps or real-imaginary exchanges in hardware, so the practical implementation cost is even lower.

2.2 Computation Time and Scaling

If the time to perform a complex multiply-accumulate is modeled as a constant, then the direct DFT has time complexity

$$T_{\text{DFT}}(N) = \mathcal{O}(N^2). \tag{18}$$

For radix-2 FFT,

$$T_{\text{FFT}}(N) = 2T\left(\frac{N}{2}\right) + cN, \tag{19}$$

where cN represents the butterfly operations and twiddle multiplications at one stage. By the Master Theorem or repeated substitution,

$$T_{\text{FFT}}(N) = \mathcal{O}(N\log_2 N). \tag{20}$$

The difference becomes dramatic for large N. For $N = 1024$,

$$N^2 = 1{,}048{,}576, \tag{21}$$

$$N \log_2 N = 1024 \cdot 10 = 10{,}240, \tag{22}$$

which shows why FFT is the preferred transform engine in digital communications, spectral analysis, radar, audio processing, and hardware accelerators.

2.3 Hardware Meaning of "Computation Time"

In hardware, "computation time" can refer to several related quantities:

- **Latency:** the time from first valid input sample to final FFT output.
- **Throughput:** how often a new transform result can be produced.
- **Critical path delay:** the longest combinational path, which determines the maximum clock frequency.

For an 8-point radix-2 FFT, the number of algorithmic stages is

$$\log_2 8 = 3. \tag{23}$$

Thus, a fully parallel pipelined hardware architecture naturally maps to three butterfly stages with intervening rotation blocks. If one pipeline register is inserted after each stage, the algorithmic latency is approximately three clock stages plus any extra cycles required by complex multipliers, while the throughput can approach one transform per frame after the pipeline fills.

3 Derivation of the Radix-2 DIF FFT

3.1 General DIF Decomposition

Starting from

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn}, \tag{24}$$

split the sum into the first and second halves of the time samples:

$$X[k] = \sum_{n=0}^{N/2-1} x[n] W_N^{kn} + \sum_{n=N/2}^{N-1} x[n] W_N^{kn}. \tag{25}$$

Let $n = m + N/2$ in the second sum. Then

$$X[k] = \sum_{m=0}^{N/2-1} x[m] W_N^{km} + \sum_{m=0}^{N/2-1} x[m + N/2] W_N^{k(m+N/2)} \tag{26}$$

$$= \sum_{m=0}^{N/2-1} x[m] W_N^{km} + \sum_{m=0}^{N/2-1} x[m + N/2] W_N^{km} W_N^{kN/2}. \tag{27}$$

Since

$$W_N^{N/2} = e^{-\mathrm{j}\pi} = -1, \tag{28}$$

we have

$$W_N^{kN/2} = (-1)^k. \tag{29}$$

Therefore,

$$X[k] = \sum_{m=0}^{N/2-1} \Big(x[m] + (-1)^k x[m+N/2] \Big) W_N^{km}. \tag{30}$$

Now separate the even-indexed and odd-indexed frequency outputs.

For even $k = 2r$,

$$X[2r] = \sum_{m=0}^{N/2-1} \left(x[m] + x[m+N/2] \right) W_N^{2rm} \tag{31}$$

$$= \sum_{m=0}^{N/2-1} \left(x[m] + x[m+N/2] \right) W_{N/2}^{rm}. \tag{32}$$

Define

$$g[m] = x[m] + x[m+N/2]. \tag{33}$$

Then

$$X[2r] = \sum_{m=0}^{N/2-1} g[m] W_{N/2}^{rm}, \tag{34}$$

which is an $(N/2)$-point DFT.

For odd outputs $k = 2r+1$,

$$X[2r+1] = \sum_{m=0}^{N/2-1} \left(x[m] - x[m+N/2] \right) W_N^{(2r+1)m} \tag{35}$$

$$= \sum_{m=0}^{N/2-1} \left(x[m] - x[m+N/2] \right) W_N^m W_{N/2}^{rm}. \tag{36}$$

Define

$$h[m] = \left(x[m] - x[m+N/2] \right) W_N^m. \tag{37}$$

Then

$$X[2r+1] = \sum_{m=0}^{N/2-1} h[m] W_{N/2}^{rm}, \tag{38}$$

which is also an $(N/2)$-point DFT.

Thus the radix-2 DIF algorithm is

$$\text{upper branch:} \quad g[m] = x[m] + x[m+N/2], \tag{39}$$

$$\text{lower branch:} \quad h[m] = \left(x[m] - x[m+N/2] \right) W_N^m, \tag{40}$$

followed by two smaller $(N/2)$-point DFTs.

3.2 Butterfly Equations

The elementary radix-2 DIF butterfly takes a pair (a, b) and produces

$$y_0 = a + b, \tag{41}$$
$$y_1 = (a - b)\, W, \tag{42}$$

where W is the appropriate twiddle factor for that butterfly. In hardware,

- the **butterfly block** performs one addition and one subtraction,
- the **rotation block** multiplies the lower branch by the twiddle factor W.

This separation is attractive because the butterfly is simple and repeated many times, while the rotation block can be optimized independently.

4 Detailed 8-Point Radix-2 DIF Derivation

For $N = 8$, the first DIF split gives

$$X[k] = \sum_{n=0}^{3} \Big(x[n] + (-1)^k x[n+4] \Big)\, W_8^{kn}. \tag{43}$$

Define the stage-1 outputs

$$s_0 = x[0] + x[4], \qquad d_0 = (x[0] - x[4])W_8^0, \tag{44}$$
$$s_1 = x[1] + x[5], \qquad d_1 = (x[1] - x[5])W_8^1, \tag{45}$$
$$s_2 = x[2] + x[6], \qquad d_2 = (x[2] - x[6])W_8^2, \tag{46}$$
$$s_3 = x[3] + x[7], \qquad d_3 = (x[3] - x[7])W_8^3. \tag{47}$$

Then the upper 4-point transform operates on s_0, s_1, s_2, s_3 and the lower 4-point transform operates on d_0, d_1, d_2, d_3.

Applying radix-2 DIF again to each 4-point block gives stage-2 quantities

$$u_0 = s_0 + s_2, \qquad v_0 = (s_0 - s_2)W_4^0 = s_0 - s_2, \tag{48}$$
$$u_1 = s_1 + s_3, \qquad v_1 = (s_1 - s_3)W_4^1 = -\mathrm{j}(s_1 - s_3), \tag{49}$$
$$u_2 = d_0 + d_2, \qquad v_2 = (d_0 - d_2)W_4^0 = d_0 - d_2, \tag{50}$$
$$u_3 = d_1 + d_3, \qquad v_3 = (d_1 - d_3)W_4^1 = -\mathrm{j}(d_1 - d_3). \tag{51}$$

The final 2-point butterflies generate stage-3 outputs

$$Y_0 = u_0 + u_1, \qquad Y_4 = u_0 - u_1, \tag{52}$$
$$Y_2 = v_0 + v_1, \qquad Y_6 = v_0 - v_1, \tag{53}$$
$$Y_1 = u_2 + u_3, \qquad Y_5 = u_2 - u_3, \tag{54}$$
$$Y_3 = v_2 + v_3, \qquad Y_7 = v_2 - v_3. \tag{55}$$

The DIF flow graph naturally produces outputs in bit-reversed order, namely

$$\big[Y_0, Y_4, Y_2, Y_6, Y_1, Y_5, Y_3, Y_7\big] = \big[X[0], X[4], X[2], X[6], X[1], X[5], X[3], X[7]\big]. \tag{56}$$

Hence, either a final reorder network is used, or the subsequent hardware block accepts bit-reversed ordering directly.

4.1 Compact Hardware Form

The 8-point DIF FFT can be implemented efficiently using a radix-2 *single-path delay feedback* (SDF) architecture. In an SDF structure, the input samples arrive as a single stream, and each stage contains one radix-2 butterfly together with a delay/feedback path that stores earlier samples until the correct butterfly pair is available. This approach is very attractive in hardware because it reuses the same butterfly and rotator hardware over time instead of instantiating a fully parallel network.

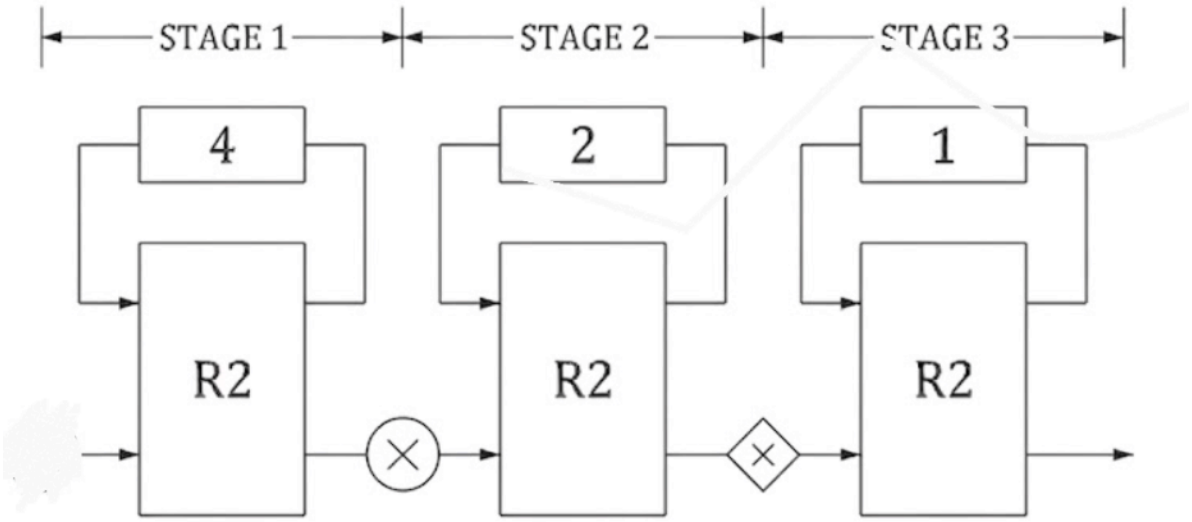


Figure 1: Radix-2 DIF SDF hardware architecture for the 8-point FFT. The `R2` blocks are butterfly units, the circles marked with $\times$ are rotator blocks, and the diamond marked with $\times$ is a direct pass-through element. The blocks labeled 4, 2, and 1 indicate the stage-by-stage delay organization of the streaming pipeline.

In Figure 1, each `R2` block performs the radix-2 butterfly operation

$$y_0 = a + b, \tag{57}$$

$$y_1 = a - b, \tag{58}$$

and when a twiddle multiplication is required, the lower branch is passed through a rotator so that the effective output becomes

$$y_1 = (a - b)W. \tag{59}$$

The feedback loops above the `R2` blocks implement the delay memories required by the SDF schedule. For an 8-point transform, the delays reduce from stage to stage, which is why the diagram is

marked with 8, 4, 2, and 1. Conceptually, the first stage handles the widest spacing between paired samples, the next stage handles smaller sub-transforms, and the final stage completes the shortest butterfly combinations.

More specifically, the first computational stage forms the pairs $(x[0], x[4])$, $(x[1], x[5])$, $(x[2], x[6])$, and $(x[3], x[7])$, corresponding to the first DIF decomposition. The next stage processes the resulting 4-point subproblems, and the next stage completes the 2-point combinations. The circular blocks marked with $\times$ are rotators that apply the required twiddle factors on the lower butterfly branch, while the diamond marked with $\times$ is a no-operation path in this design and simply forwards the data to the output stage.

Hence the hardware cost is dominated by repeated add/subtract units, small delay memories, and a small number of twiddle multipliers. For $N = 8$, several twiddle factors are trivial:

$$W_8^0 = 1, \tag{60}$$

$$W_8^2 = -\mathrm{j}, \tag{61}$$

$$W_4^0 = 1, \tag{62}$$

$$W_4^1 = -\mathrm{j}. \tag{63}$$

Multiplication by $-\mathrm{j}$ is especially cheap in hardware because for a complex number $a + b\mathrm{j}$,

$$-\mathrm{j}(a + b\mathrm{j}) = b - a\mathrm{j}, \tag{64}$$

which is only a swap and sign inversion. Because SDF processes one sample stream through a deeply regular pipeline, it provides a good balance between hardware reuse, throughput, and implementation simplicity.

Note that a **reorder buffer** with a ping pong memory has been used to get the full effect of the pipeline when multiple signals are passed through one after the another and also we get the output in ordered format.

4.2 SDF Throughput and Latency

The radix-2 SDF architecture provides a practical balance between throughput and hardware cost. Instead of using many butterflies in parallel, SDF reuses a small number of butterfly and rotator blocks while samples move through delay-feedback paths. As a result, the design is area-efficient and still supports continuous streaming operation.

For a properly scheduled SDF pipeline, one input sample can be accepted every clock cycle. Therefore, the steady-state sample throughput is

$$R_{\text{sample}} = 1 \text{ sample/clock}. \tag{65}$$

Since one transform contains $N = 8$ samples, the transform throughput is

$$R_{\text{FFT}} = \frac{1}{8} \text{ FFT/clock}, \tag{66}$$

which means one complete 8-point FFT result is produced every eight clock cycles after the pipeline reaches steady state.

Latency is different from throughput. In SDF, latency comes mainly from the delay elements needed to align the streaming samples before each butterfly operation. For the 8-point radix-2 case, the stage delays follow the sequence

$$D_1 = 4, \qquad D_2 = 2, \qquad D_3 = 1. \tag{67}$$

If the butterfly and rotator stages are pipelined, a simple latency model is

$$L_{\text{total}} \approx D_1 + D_2 + D_3 + L_{\text{arith}}, \tag{68}$$

where L_{arith} is the arithmetic pipeline depth.

Thus, the SDF FFT offers high streaming throughput, but its latency is not minimal because some samples must wait inside the feedback delays before pairing. This trade-off is often acceptable in hardware DSP systems, since SDF greatly reduces area while preserving regularity and continuous data flow.

5 Signal-by-Signal 8-Point FFT Results

Throughout this section, let

$$N = 8, \qquad j = \{0, 1, 2, 3, 4, 5, 6, 7\}. \tag{69}$$

The five test signals are

$$\text{DC:} \quad x[n] = 0.05, \tag{70}$$

$$\text{Nyquist:} \quad x[n] = 0.05(-1)^n, \tag{71}$$

$$\text{Single Sine:} \quad x[n] = 0.05 \sin\left(\frac{2\pi n}{8}\right), \tag{72}$$

$$\text{Fast Cosine:} \quad x[n] = 0.05 \cos\left(\frac{4\pi n}{8}\right), \tag{73}$$

$$\text{Impulse:} \quad x[n] = 0.05\,\delta[n]. \tag{74}$$

The detailed derivation of the radix-2 DIF FFT has already been presented in the previous sections. For the test signals below, only the input sequences and final 8-point FFT results are retained.

5.1 Case 1: Constant DC Input

The input is

$$x[n] = 0.05, \qquad n = 0, 1, \ldots, 7. \tag{75}$$

Therefore the time-domain vector is

$$\boldsymbol{x}_{\text{DC}} = \begin{bmatrix} 0.05 & 0.05 & 0.05 & 0.05 & 0.05 & 0.05 & 0.05 & 0.05 \end{bmatrix}^T. \tag{76}$$

The FFT output is

$$X[k] = \begin{cases} 0.4, & k = 0, \\ 0, & k = 1, 2, \ldots, 7. \end{cases} \tag{77}$$

Thus all energy is concentrated in the DC bin.

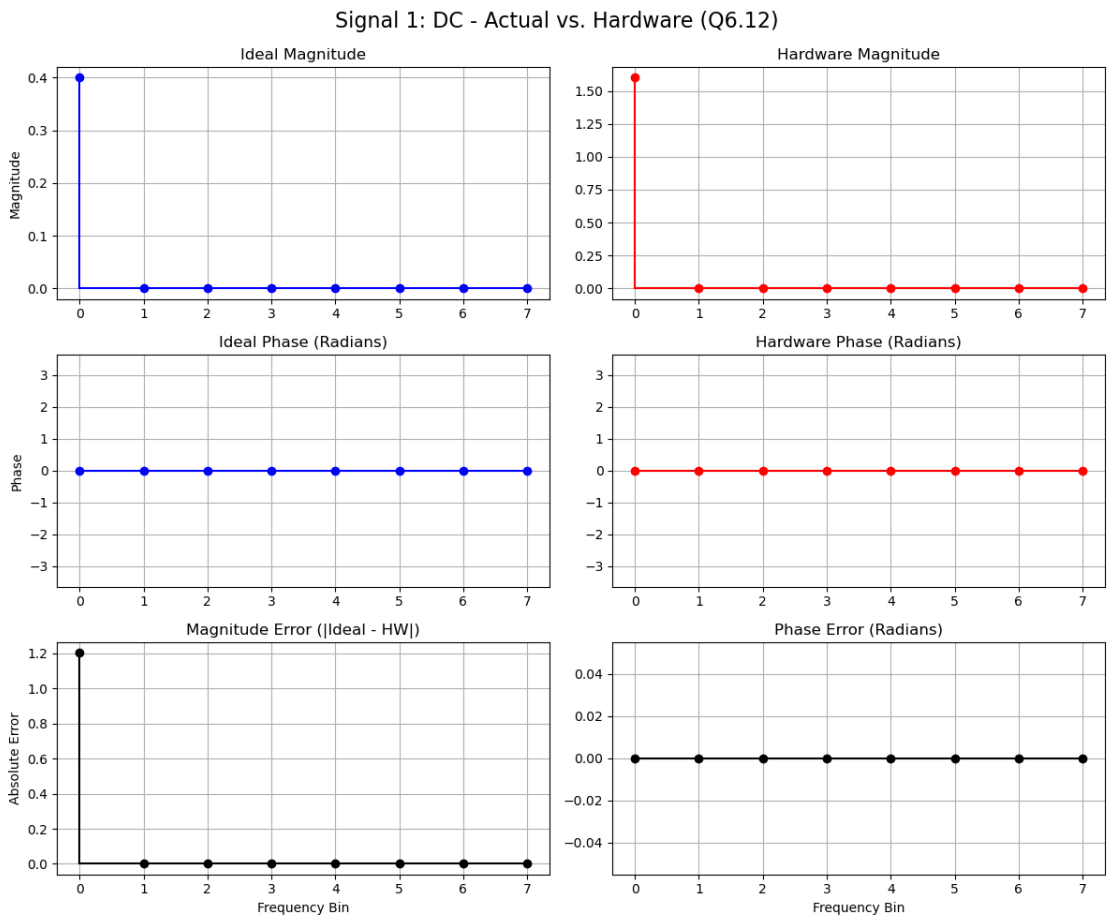


Figure 2: DC input waveform and corresponding 8-point FFT result.

5.2 Case 2: Nyquist Alternating Input

The input is

$$x[n] = 0.05(-1)^n. \tag{78}$$

Hence

$$\boldsymbol{x}_{\text{Nyq}} = \begin{bmatrix} 0.05 & -0.05 & 0.05 & -0.05 & 0.05 & -0.05 & 0.05 & -0.05 \end{bmatrix}^T. \tag{79}$$

The FFT output is

$$X[k] = \begin{cases} 0.4, & k = 4, \\ 0, & k \neq 4. \end{cases} \tag{80}$$

Thus the Nyquist bin alone is active.

5.3 Case 3: Single Sine Wave

The input is

$$x[n] = 0.05 \sin\left(\frac{2\pi n}{8}\right) = 0.05 \sin\left(\frac{\pi n}{4}\right). \tag{81}$$

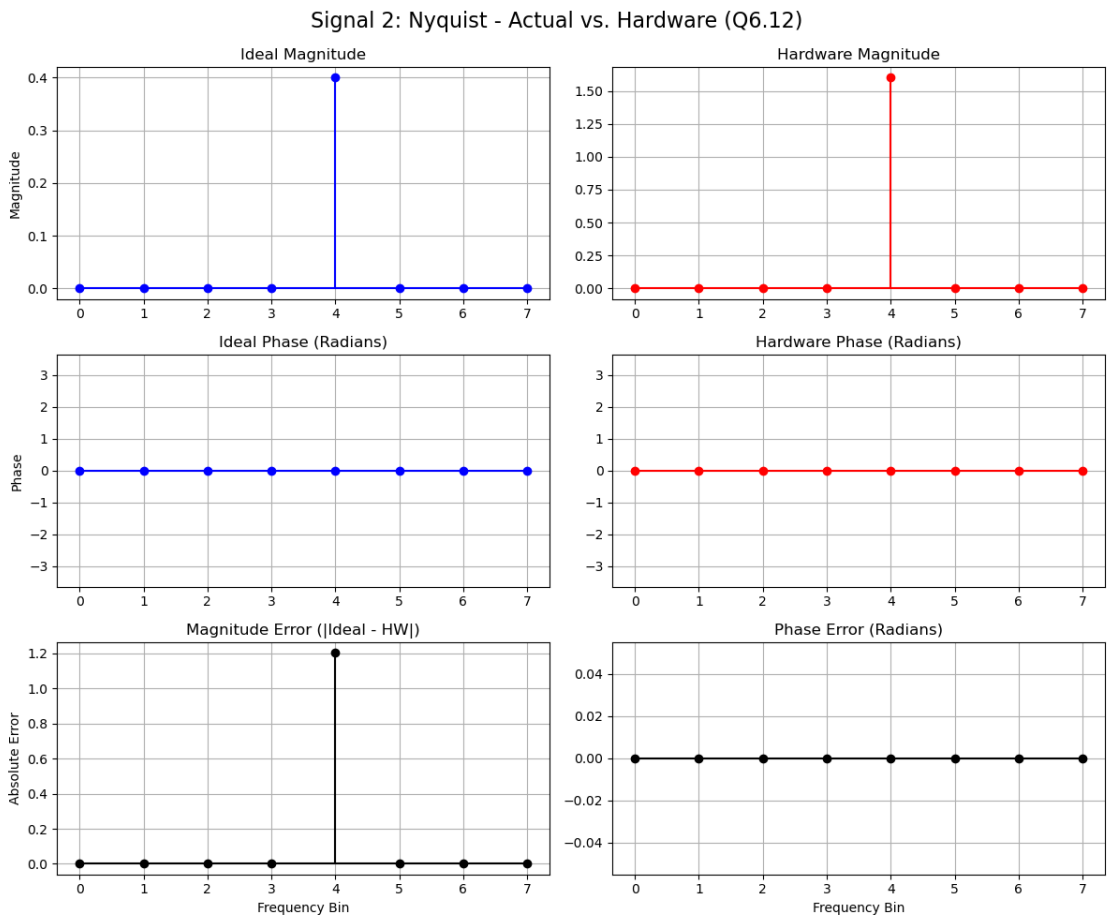


Figure 3: Nyquist alternating input waveform and corresponding 8-point FFT result.

The samples are

$$\boldsymbol{x}_{\sin} = \left[0, \tfrac{0.05}{\sqrt{2}}, 0.05, \tfrac{0.05}{\sqrt{2}}, 0, -\tfrac{0.05}{\sqrt{2}}, -0.05, -\tfrac{0.05}{\sqrt{2}}\right]^T. \tag{82}$$

The FFT output is

$$X[k] = \begin{cases} -0.2\text{j}, & k = 1, \\ 0.2\text{j}, & k = 7, \\ 0, & \text{otherwise.} \end{cases} \tag{83}$$

Thus the signal occupies the conjugate frequency bins associated with a single real sinusoid.

The extra peak that happened in the this sine graph (verilog) is because of the quantization nosie.

5.4 Case 4: Faster Cosine Wave

The input is

$$x[n] = 0.05 \cos\left(\frac{4\pi n}{8}\right) = 0.05 \cos\left(\frac{\pi n}{2}\right). \tag{84}$$

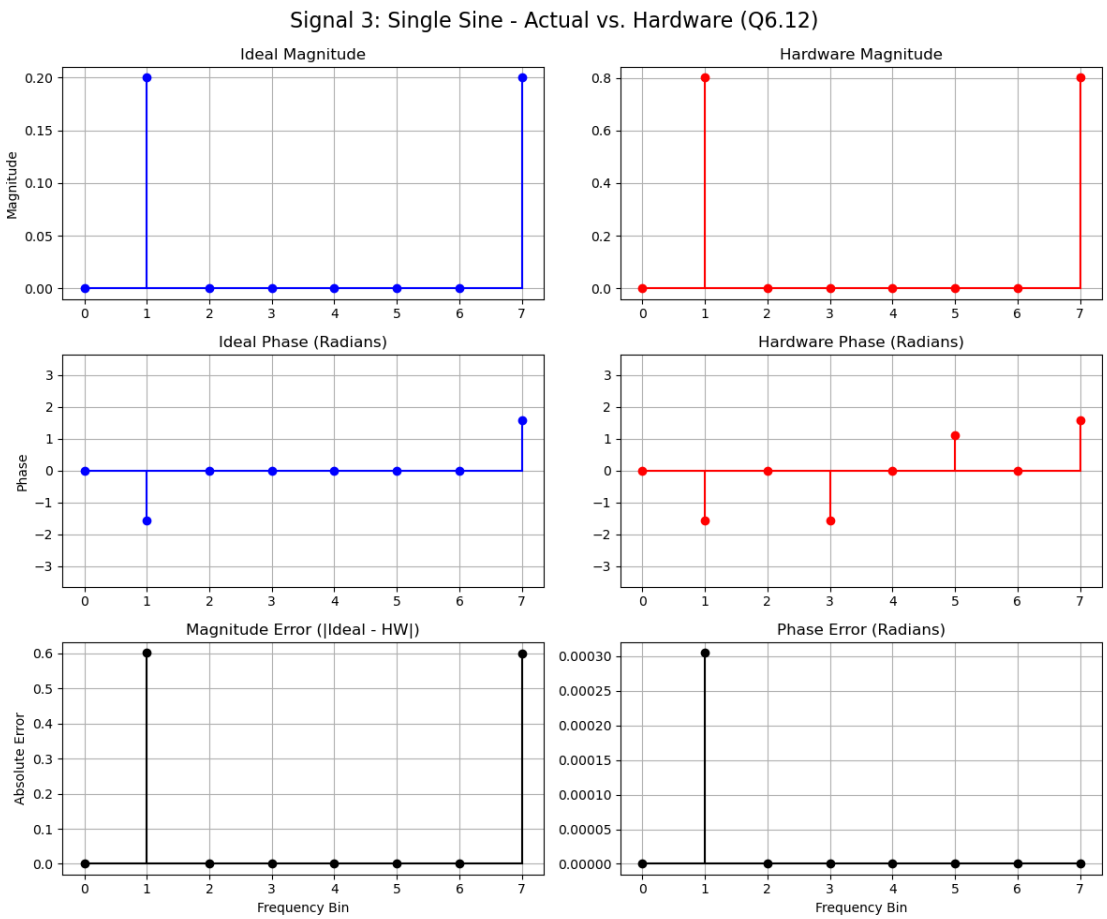


Figure 4: Single-sine input waveform and corresponding 8-point FFT result.

The corresponding sample sequence is

$$\boldsymbol{x}_{\cos} = \begin{bmatrix} 0.05 & 0 & -0.05 & 0 & 0.05 & 0 & -0.05 & 0 \end{bmatrix}^T. \tag{85}$$

The FFT output is

$$X[k] = \begin{cases} 0.2, & k = 2, \\ 0.2, & k = 6, \\ 0, & \text{otherwise.} \end{cases} \tag{86}$$

Thus the cosine produces two equal real-valued spectral lines.

5.5 Case 5: Impulse Input

The input is

$$x[n] = 0.05\,\delta[n], \tag{87}$$

so the sample vector is

$$\boldsymbol{x}_{\delta} = \begin{bmatrix} 0.05 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}^T. \tag{88}$$

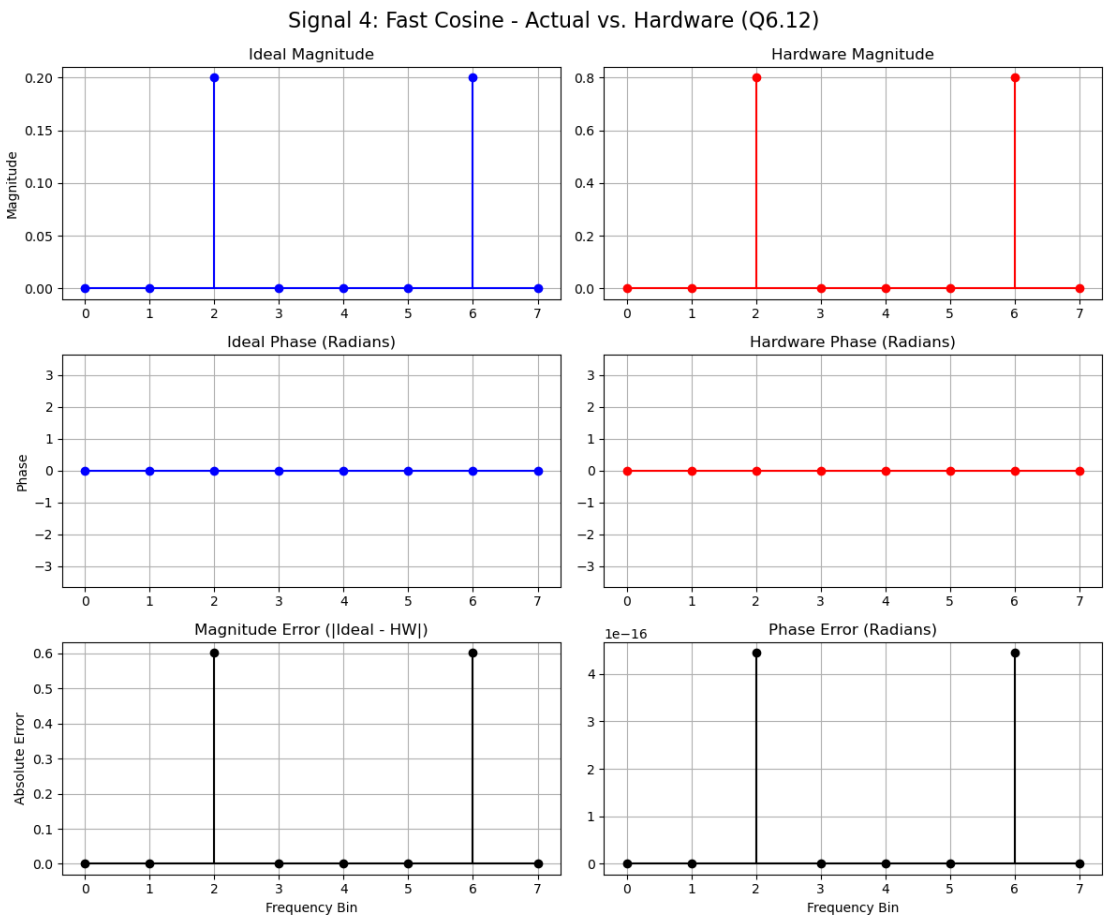


Figure 5: Fast-cosine input waveform and corresponding 8-point FFT result.

The FFT output is uniform across all bins:

$$X[0] = X[1] = X[2] = X[3] = X[4] = X[5] = X[6] = X[7] = 0.05. \tag{89}$$

Thus the impulse has a flat spectrum.

6 Quartus resources and fmax

7 Quartus resource and fmax

8 FFT IP Intel quartus

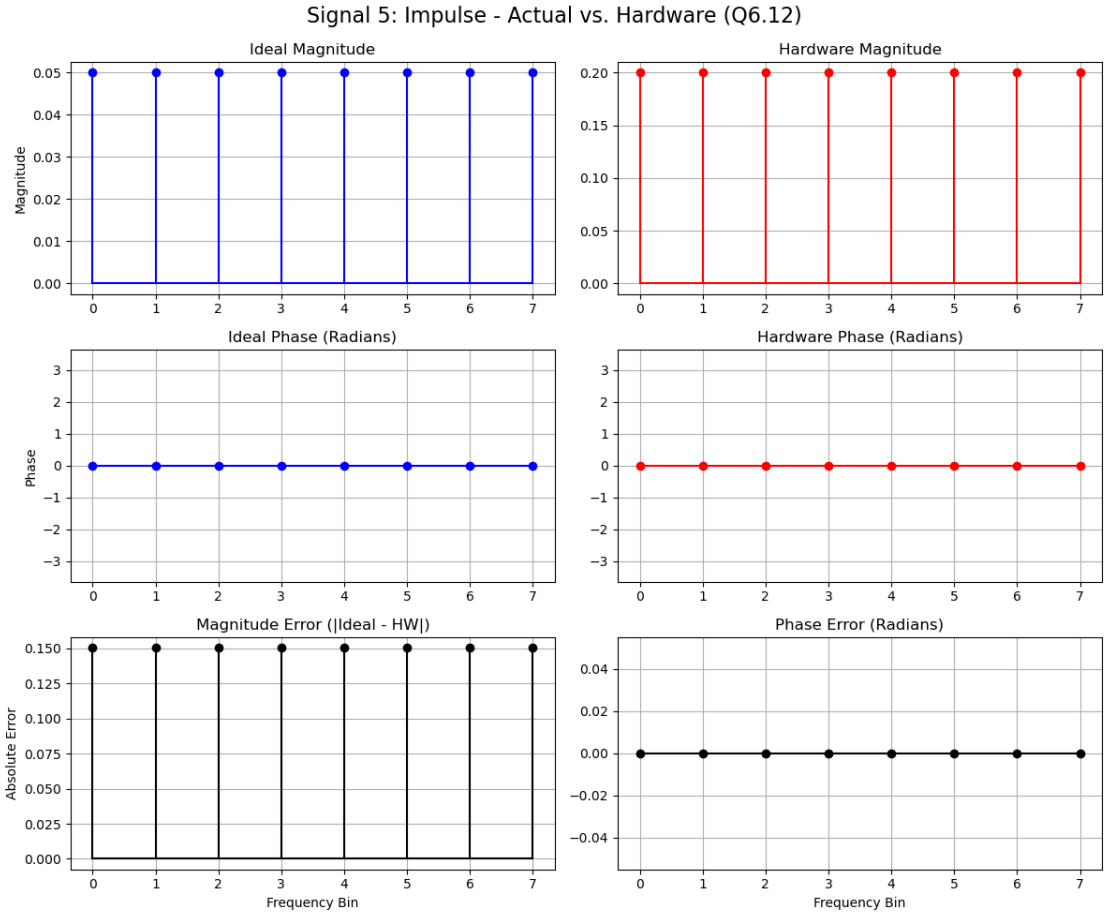


Figure 6: Impulse input waveform and corresponding 8-point FFT result.

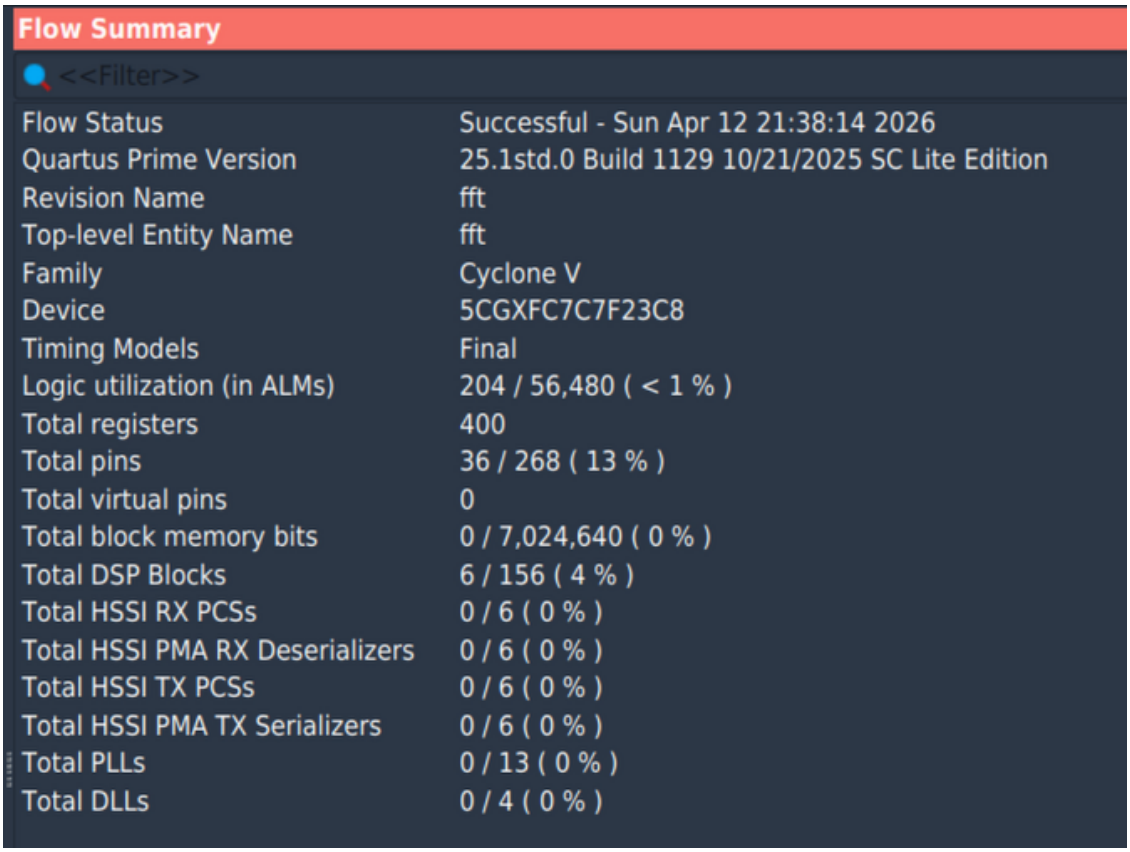


Figure 7: Quartus resource utilization for the FFT hardware implementation.

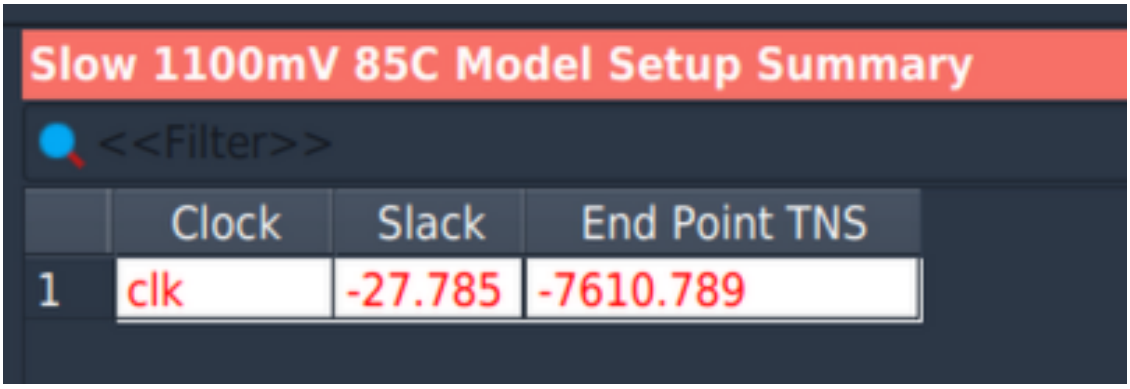


Figure 8: Quartus setup timing report for the FFT hardware implementation.

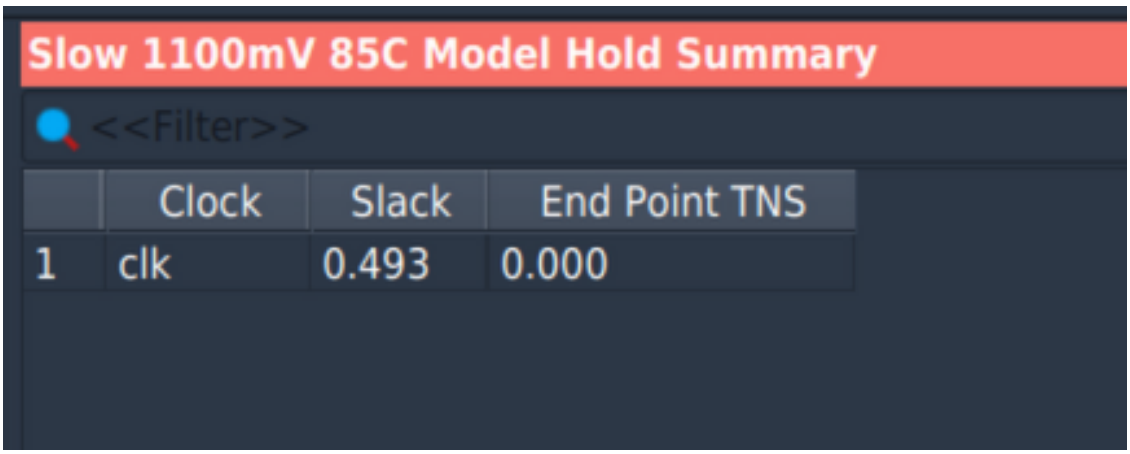


Figure 9: Quartus hold timing report for the FFT hardware implementation.

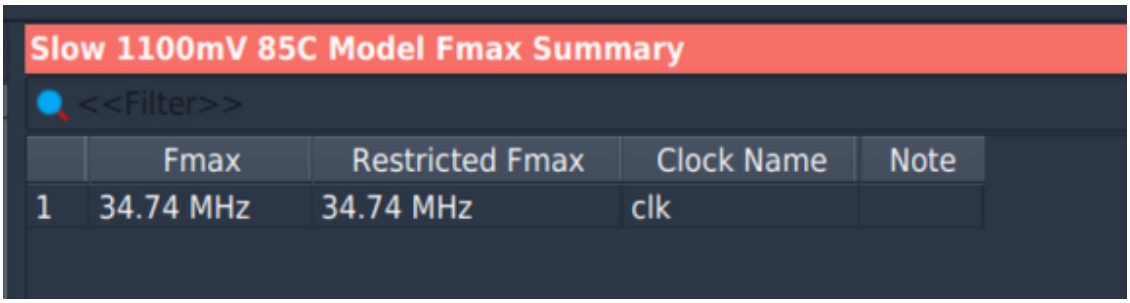


Figure 10: Quartus Fmax report for the FFT hardware implementation.

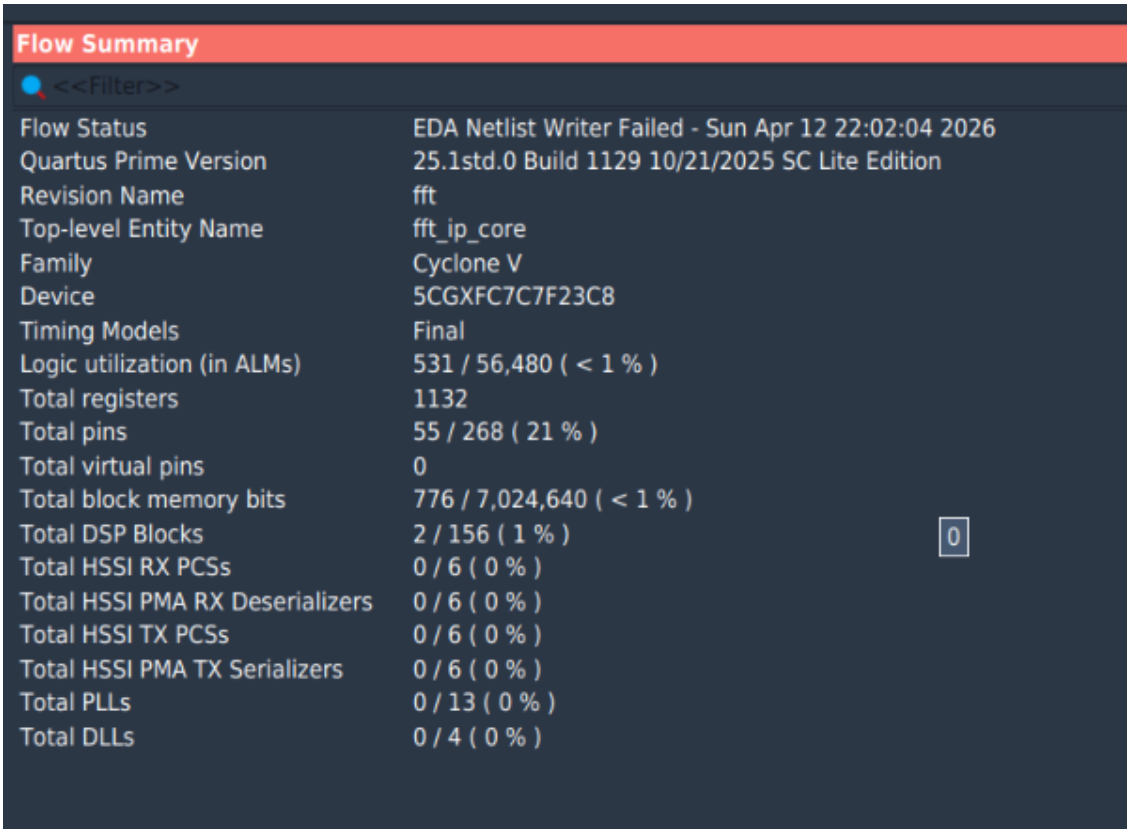


Figure 11: Quartus resource report for the FFT Intel IP.

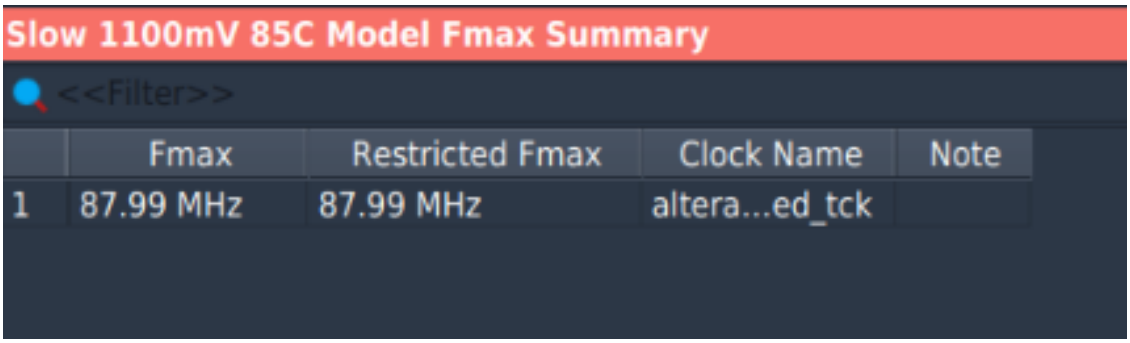


Figure 12: Quartus Fmax report for the FFT Intel IP.